Pemahaman dari Modul dan Kode Projek Alpro Muhammad Aziz Syah Dani

---

Modul A

Alur Penerimaan Barang

Modul A berfokus pada logika dasar bagaimana gudang menerima barang baru dan mencatatnya dalam sistem. Alur utama dimulai ketika barang datang, kemudian dilakukan pemeriksaan fisik untuk memastikan kondisi barang masih layak. Jika kondisi tidak baik maka barang ditolak dan proses selesai. Jika kondisi baik maka petugas menginput data barang, sistem menghasilkan id resi yang unik, lalu seluruh data disimpan ke dalam struktur data.

Flowchart pada modul ini menggambarkan urutan langkah berikut. Mulai, terima barang, cek fisik, keputusan apakah barang layak, jika tidak layak maka barang ditolak dan proses berakhir. Jika layak maka petugas menginput data barang, sistem melakukan generate id resi unik, kemudian data disimpan dan proses selesai. Dengan alur seperti ini dosen bisa melihat bahwa logika dasar yang diminta di soal sudah tercakup seluruhnya.

Pseudocode fungsi `input_barang` menjelaskan langkah algoritmik secara terstruktur. Pertama program meminta input kondisi fisik barang. Jika jawaban bukan baik atau bagus maka fungsi langsung menolak barang dan selesai. Jika kondisi baik maka program meminta seluruh data penting yaitu kode barang, nama barang, harga, tahun kedaluwarsa, nomor rak, tanggal masuk, dan jumlah. Setelah itu fungsi menghitung id resi baru. Caranya dengan mengambil nilai id resi terbesar yang sudah ada di data lalu menambah satu. Terakhir fungsi membuat objek barang baru dan menambahkannya ke list data. Pesan keberhasilan akan ditampilkan agar pengguna tahu id resi yang baru terbentuk.

Implementasi di Python tetap mengikuti paradigma prosedural. Data barang diwakili oleh satu struktur `Barang` dengan beberapa field. Fungsi `load_data` bertanggung jawab membaca file `data_barang.csv` dan mengubah setiap baris menjadi objek `Barang`. Fungsi `input_barang` bekerja di atas list barang tersebut. Struktur program mudah dibaca, variabel memiliki nama jelas, dan setiap langkah yang ada di flowchart dan pseudocode sudah diwujudkan dalam kode. Hal ini memenuhi aspek logika dasar, representasi algoritma, dan kualitas kode pada rubrik penilaian.

Modul B

Analisis Efisiensi Bubble Sort dan Quick Sort

Modul B menekankan kemampuan analitis mahasiswa terhadap efisiensi algoritma. Dua algoritma sorting yang dibandingkan adalah Bubble Sort dan Quick Sort. Keduanya digunakan untuk mengurutkan barang berdasarkan harga, sehingga hasil runtime dan teori Big O dapat langsung dihubungkan dengan studi kasus gudang yang memproses banyak data.

Secara konsep, Bubble Sort bekerja dengan membandingkan elemen saling berdekatan lalu menukarnya jika urutan salah. Satu kali putaran akan menempatkan elemen terbesar di bagian akhir. Proses ini diulang terus sampai tidak ada lagi pertukaran. Untuk n data, jumlah perbandingan tumbuh secara kuadrat sehingga pada kasus rata rata dan terburuk kompleksitas waktunya adalah $O(n^2)$. Pada kasus terbaik, bila data sudah terurut dan program diberi pengecekan apakah masih ada pertukaran, algoritma hanya perlu satu kali lewat sehingga mendekati O(n). Ruang tambahan yang diperlukan hanya beberapa variabel penampung sehingga kompleksitas ruang adalah O(1).

Quick Sort bekerja dengan ide pembagian. Algoritma memilih satu nilai sebagai pivot lalu membagi data menjadi tiga bagian yaitu data yang lebih kecil dari pivot, sama dengan pivot, dan lebih besar dari pivot. Setelah itu fungsi Quick Sort dipanggil kembali secara rekursif untuk bagian kiri dan kanan. Jika pembagian cukup seimbang maka pada setiap tingkat rekursi jumlah elemen yang diproses berkurang setengah, sehingga kompleksitas waktu rata rata dan terbaik adalah O(n log n). Namun pada kasus terburuk, ketika pivot selalu sangat tidak seimbang, misalnya selalu elemen paling kecil, algoritma dapat turun menjadi $O(n^2)$. Kompleksitas ruang berasal dari tumpukan rekursi sekitar O(log n) pada kasus normal.

Kode `modul_b.py` tidak hanya mengimplementasikan kedua algoritma tetapi juga melakukan pengukuran runtime menggunakan modul `time`. Program memuat data barang dari file csv, kemudian mencatat waktu sebelum dan sesudah pemanggilan Bubble Sort dan Quick Sort. Selisih waktu tersebut ditampilkan sebagai runtime untuk masing masing algoritma. Dengan demikian analisis tidak berhenti pada teori Big O, tetapi juga menyajikan data eksperimen yang menggambarkan perbedaan performa di kasus nyata. Hal ini sesuai dengan rubrik yang menekankan ketelitian analisis Big O dan validitas data eksperimen untuk membandingkan efisiensi algoritma.

Modul C

Pengurutan Inventaris dengan Bubble Sort, Selection Sort, dan Quick Sort

Modul C berhubungan langsung dengan kebutuhan operasional gudang yaitu menyiapkan laporan dalam urutan tertentu. Ada tiga kebutuhan utama. Pertama, mengurutkan barang berdasarkan harga termurah. Kedua, mengurutkan barang berdasarkan tahun kedaluwarsa agar barang yang hampir habis masa pakainya bisa diprioritaskan. Ketiga, mengurutkan barang berdasarkan id resi sebagai dasar pencarian cepat pada modul berikutnya.

Bubble Sort dipakai khusus untuk belajar konsep dasar dan menunjukkan contoh algoritma sederhana yang kurang efisien. Pada fungsi `bubble_sort_by_harga`, data barang disalin lalu diurutkan berdasarkan field harga. Setiap iterasi luar melakukan beberapa perbandingan di bagian dalam. Jika tidak ada pertukaran dalam satu putaran, algoritma berhenti karena data sudah terurut. Walaupun lambat untuk data besar, implementasi ini sangat jelas dan mudah diikuti, sehingga cocok untuk menjelaskan urutan langkah ke dosen.

Algoritma Selection Sort digunakan untuk menyusun barang berdasarkan tahun kedaluwarsa. Pertimbangannya adalah selection sort meminimalkan jumlah pertukaran. Dalam setiap iterasi luar, algoritma hanya mencari indeks elemen dengan tahun kedaluwarsa paling kecil di bagian yang belum terurut. Setelah itu dilakukan satu kali pertukaran dengan posisi awal. Jumlah swap yang sedikit ini sejalan dengan ide mengurangi biaya pemindahan data. Di konteks gudang, hal tersebut dapat dianalogikan sebagai upaya meminimalkan perpindahan fisik atau biaya update data yang besar.

Quick Sort dipakai untuk mengurutkan barang berdasarkan id resi. Fungsi `quick_sort_by_id` menggunakan rekursi secara eksplisit. Jika panjang data satu atau nol, data langsung dikembalikan. Bila lebih dari itu, algoritma memilih pivot berupa id resi di posisi tengah, lalu membentuk tiga list baru yaitu bagian kiri, tengah, dan kanan. Fungsi yang sama dipanggil lagi untuk bagian kiri dan kanan, kemudian hasilnya digabung. Bentuk ini jelas menggambarkan prinsip divide and conquer dan penggunaan rekursi, sehingga memenuhi syarat rubrik bahwa Quicksort harus didemonstrasikan dengan rekursi.

Di akhir `modul_c.py`, program menampilkan beberapa barang pertama dari tiap hasil pengurutan. Bagian ini dapat dipakai sebagai bukti bahwa sorting berjalan benar dan sesuai kriteria. Dengan demikian seluruh algoritma sorting wajib sudah diimplementasikan, struktur kode rapi, dan penjelasan pemilihan Selection Sort untuk expired date juga sudah jelas.

Modul D

Pencarian Barang dengan Linear Search dan Binary Search

Modul D menguji pemahaman mahasiswa tentang algoritma searching dan keterkaitannya dengan kondisi data. Ada dua jenis pencarian yang harus dibuat, yaitu berdasarkan nama barang dan berdasarkan id resi.

Pencarian berdasarkan nama barang menggunakan Linear Search. Nama barang pada gudang bersifat teks bebas dan tidak selalu tersusun dalam urutan tertentu, bahkan bisa memiliki kata yang mirip. Fungsi `linear_search_by_nama` menelusuri seluruh list barang satu per satu. Setiap nama barang diubah ke huruf kecil, kemudian dicek apakah mengandung kata kunci yang dimasukkan pengguna. Semua barang yang cocok dikumpulkan dalam list hasil dan dikembalikan. Karena mengecek satu per satu, kompleksitas waktunya O(n). Meskipun demikian, cara ini fleksibel dan cocok untuk situasi di mana urutan data tidak terjamin.

Pencarian berdasarkan id resi menggunakan Binary Search. Syarat pentingnya adalah data harus sudah terurut terlebih dahulu berdasarkan id resi. Hal ini dipenuhi dengan memanggil `quick_sort_by_id` dari modul C. Setelah data terurut, fungsi `binary_search_by_id` melakukan pencarian dengan cara membagi dua area pada setiap langkah. Nilai id resi di tengah dibandingkan dengan target. Jika sama maka barang ditemukan. Jika kurang dari target, pencarian dilanjutkan di bagian kanan. Jika lebih besar, pencarian dilanjutkan di bagian kiri. Proses berhenti ketika rentang indeks habis atau barang ditemukan. Secara teori, kompleksitas waktu pencarian ini adalah O(log n), jauh lebih cepat daripada linear search untuk data besar.

`modul_d.py` menyediakan menu sederhana agar dosen bisa mencoba kedua metode pencarian. Ketika pengguna memilih pencarian nama, program memanggil linear search dan menampilkan semua barang yang cocok. Ketika pengguna memilih pencarian id resi, program terlebih dahulu mengurutkan data dengan Quick Sort, kemudian menjalankan Binary Search. Dengan desain seperti ini, modul D memenuhi rubrik karena kedua algoritma pencarian wajib sudah terimplementasi dan prasyarat data terurut untuk Binary Search juga dihormati.

Modul E

Penyimpanan Cepat dengan Hash Table

Modul E memanfaatkan struktur data yang lebih canggih yaitu Hash Table agar pencarian lokasi rak bisa dilakukan hampir instan. Di gudang yang besar, jumlah rak bisa mencapai ribuan. Jika setiap kali mencari nomor rak harus melakukan loop ke seluruh data barang maka waktu respon akan sangat lama. Hash Table mengatasi persoalan ini dengan memetakan kode barang langsung ke indeks tertentu pada array.

Kelas `HashTableLokasiRak` memiliki sebuah array bucket dengan ukuran tertentu. Fungsi `_hash` mengubah kode barang yang berupa string menjadi sebuah indeks dengan cara menjumlahkan nilai karakter lalu mengambil sisa bagi terhadap ukuran tabel. Pendekatan ini cukup sederhana tetapi sudah cukup untuk mendistribusikan kode barang ke berbagai bucket. Saat fungsi `insert` dipanggil, struktur akan mencari bucket berdasarkan hasil hash. Jika kode barang tersebut sudah ada, nomor rak diperbarui. Jika belum ada, pasangan kode barang dan nomor rak ditambahkan ke bucket itu. Metode ini disebut separate chaining karena satu indeks dapat menyimpan beberapa entri dalam bentuk list kecil.

Untuk pencarian, fungsi `get` menerima kode barang, menghitung indeks hash, lalu menelusuri list pada bucket tersebut untuk mencari kode yang sesuai. Jika ditemukan, fungsi langsung mengembalikan nomor rak. Jika tidak ditemukan, fungsi mengembalikan nilai kosong. Karena operasi pencarian hanya mengakses beberapa elemen pada satu bucket, kompleksitas waktu rata rata mendekati O(1). Ini jauh lebih cepat dibandingkan linear search yang memerlukan O(n) waktu.

Fungsi `build_hash_table` membangun Hash Table dari seluruh data barang yang sudah dibaca dari file csv. `modul_e.py` kemudian memberikan antarmuka sederhana. Pengguna cukup memasukkan kode barang, dan program akan menampilkan nomor rak yang terkait. Dengan demikian implementasi memenuhi rubrik untuk struktur data canggih yaitu penggunaan Hash Table untuk lookup mendekati O(1) dan integrasi dengan data simulasi gudang.
